

Instituto Federal De Educação, Ciência e Tecnologia de Rondonia

Análise e Desenvolvimento de Sistemas — 5º período

Luiz Guilherme Ribeiro da Costa

Eduarda Vitória

João do Monte

Matheus Henrique

RELATÓRIO TÉCNICO DE TESTES AUTOMATIZADOS SISTEMA AURACUP

Qualidade e Teste de Software

Porto Velho/RO

2026

SUMÁRIO

1 INTRODUÇÃO	3
2 AMBIENTE DE EXECUÇÃO DO SISTEMA	3
3 PLANO DE TESTES	3
4 ESTRATÉGIA DE AUTOMAÇÃO	4
5 EXECUÇÃO E RESULTADOS	4
6 FALHAS ENCONTRADAS E CORREÇÕES	10
7 CONCLUSÃO	11
REFERÊNCIAS	12

1 INTRODUÇÃO

Este relatório apresenta o planejamento, a automação, a execução e a análise dos testes aplicados ao sistema AuraCup, um cardápio digital de cafeteria com fluxo de pedidos e área restrita para atendentes. O sistema é composto por um frontend em React/Vite e por uma API REST desenvolvida em PHP com o framework Laravel, com persistência em banco de dados SQLite.

O objetivo da atividade não foi desenvolver novas funcionalidades, mas validar comportamentos do sistema já implementado por meio de testes automatizados, cobrindo funcionalidades, validação de dados, fluxos de navegação, integração com banco de dados e segurança básica. A automação da interface foi realizada com a ferramenta Playwright e a validação da API e do banco com PHPUnit.

2 AMBIENTE DE EXECUÇÃO DO SISTEMA

O sistema é executado localmente com as seguintes ferramentas e configurações:

- a) frontend: Vite em `http://127.0.0.1:5174` (modo de teste) ou `npm run dev` para uso geral;
- b) backend: Laravel via `php artisan serve` em `http://127.0.0.1:8000`;
- c) banco de dados: SQLite, arquivo `backend/database/database.sqlite`, criado e populado com `php artisan migrate:fresh --seed` (migrations e MenuSeeder substituem o arquivo SQL tradicional);
- d) conexão configurada em `backend/.env` (`DB_CONNECTION=sqlite`);
- e) testes: `npm run test:e2e` (Playwright, que sobe o Vite automaticamente) e `npm run test:backend` (PHPUnit com banco em memória).

As instruções completas de preparação do ambiente estão no arquivo `docs/guia-testes.md` do repositório.

3 PLANO DE TESTES

O plano de testes completo, com identificador, funcionalidade, objetivo, tipo, dados de entrada, resultado esperado, prioridade e status final, está documentado em `docs/plano-testes.md`. O Quadro 1 resume os onze casos de teste.

Quadro 1 — Resumo dos casos de teste

ID	Funcionalidade	Tipo	Resultado esperado	Status
----	----------------	------	--------------------	--------

CT01	Cardápio	Funcional / E2E	Produtos visíveis ao abrir o menu	Aprovado
CT02	Pedido (fluxo completo)	Funcional / E2E	Pedido criado e confirmado com número	Aprovado
CT03	Validação de mesa inválida	Validação / Negativo	Envio bloqueado, sem requisição à API	Aprovado
CT04	Segurança (XSS)	Segurança / E2E	Script não executa; texto escapado	Aprovado
CT05	Login de atendente	Autenticação / E2E	Área restrita liberada com pedidos listados	Aprovado
CT06	Acesso indevido	Segurança / Autorização	Login negado com mensagem de erro	Aprovado
CT07	API de pedidos	Validação / API	Payload inválido rejeitado com 422	Aprovado
CT08	Cálculo de total	Integração / Banco	Total calculado pelos preços reais	Aprovado
CT09	Login não autorizado	Segurança / API	Usuário comum recebe 401	Aprovado
CT10	Gestão de pedidos	Funcional / API	Atendente edita, conclui e cancela pedido	Aprovado
CT11	Validação de mesa vazia	Validação / Negativo	Campo obrigatório bloqueia envio	Aprovado

Fonte: elaborado pelos autores (2026).

4 ESTRATÉGIA DE AUTOMAÇÃO

A cobertura foi dividida em duas camadas complementares. A camada de interface, automatizada com Playwright (casos CT01 a CT06 e CT11), valida a experiência do usuário: abertura do sistema com `page.goto()`, preenchimento de campos com `fill()`, cliques com `click()`, navegação entre telas e validações com `expect()`. Os testes estão organizados por funcionalidade nos arquivos `cardapio.spec.js`, `pedido.spec.js` e `seguranca.spec.js`.

A camada de API e banco, automatizada com PHPUnit (casos CT07 a CT10), valida as rotas `GET /api/categories`, `GET /api/products`, `POST /api/orders` e a área de atendentes, incluindo o cálculo do total do pedido a partir dos preços reais, a persistência nas tabelas `orders` e `order_items` e a autorização por token.

Nos testes E2E, as respostas da API são interceptadas com `page.route()` para tornar os cenários determinísticos; a integração real com o banco de dados é exercitada pela camada PHPUnit, que executa as migrations e o seeder em banco SQLite em memória a cada teste.

5 EXECUÇÃO E RESULTADOS

As duas suítes foram executadas integralmente, com aprovação em todos os casos: a suíte Playwright registrou 7 testes aprovados (Figura 1) e a suíte PHPUnit registrou 10 testes aprovados com 43 asserções (Figura 2).

Figura 1 — Execução da suíte Playwright no terminal (7 testes aprovados)

```
npm test -- testes-playwright (Playwright)

> test
> playwright test

Running 7 tests using 1 worker

  ✓ 1 tests/cardapio.spec.js:4:1 > abre o menu e mostra os produtos (5.7s)
  ✓ 2 tests/pedido.spec.js:4:1 > envia um pedido e exibe a confirmacao (16.5s)
  ✓ 3 tests/pedido.spec.js:70:1 > bloqueia envio de pedido com numero de mesa invalido (12.4s)
  ✓ 4 tests/pedido.spec.js:98:1 > bloqueia envio de pedido sem numero de mesa (11.2s)
  ✓ 5 tests/seguranca.spec.js:4:1 > nao executa conteudo malicioso vindo do cardapio (2.5s)
  ✓ 6 tests/seguranca.spec.js:35:1 > abre a area restrita e lista pedidos para atendentes (14.9s)
  ✓ 7 tests/seguranca.spec.js:58:1 > bloqueia acesso de atendente com credenciais invalidas (13.4s)

7 passed (1.3m)
```

Fonte: elaborada pelos autores (2026).

Figura 2 — Execução da suíte PHPUnit no terminal (10 testes aprovados)

```
php artisan test -- backend (PHPUnit)

PASS Tests\Feature\AttendantApiTest
  ✓ atendente consegue fazer login e receber token 0.20s
  ✓ usuario comum nao consegue entrar na area de atendente 0.02s
  ✓ lista pedidos priorizando pendentes e ordem de chegada 0.03s
  ✓ atendente consegue editar concluir e cancelar pedido 0.03s

PASS Tests\Feature\MenuApiTest
  ✓ lista as categorias cadastradas no cardapio 0.04s
  ✓ lista os produtos cadastrados no cardapio 0.01s
  ✓ salva produto com campos traduzidos 0.01s

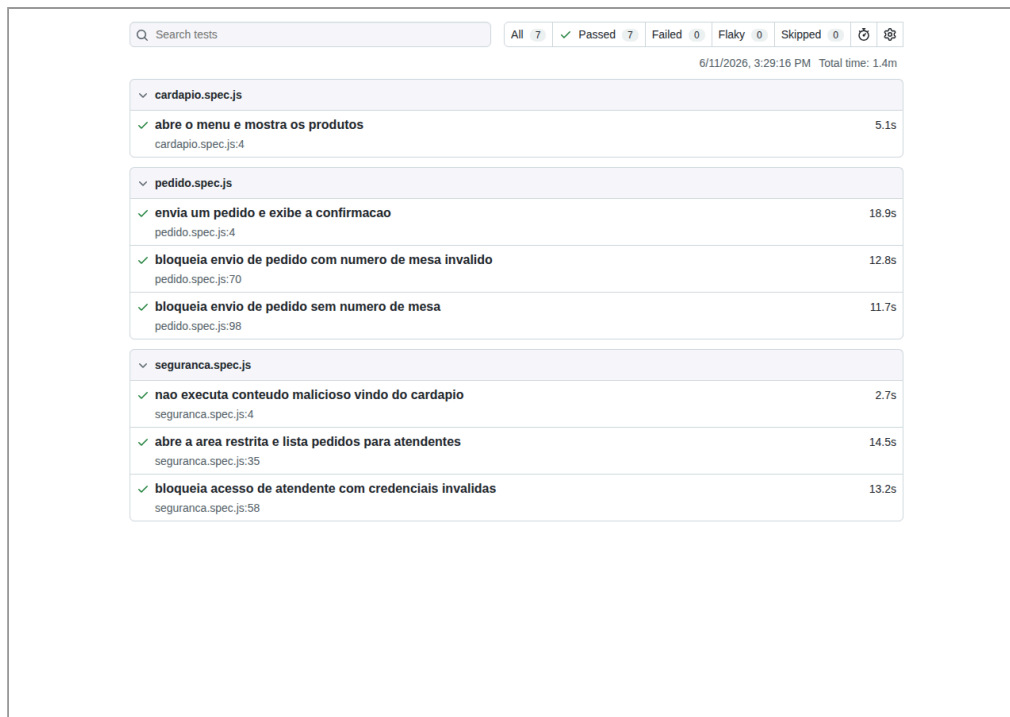
PASS Tests\Feature\OrderApiTest
  ✓ cria pedido com produtos e total correto 0.02s
  ✓ consulta pedido existente com os produtos 0.01s
  ✓ rejeita criacao de pedido com dados invalidos 0.01s

Tests: 10 passed (43 assertions)
Duration: 0.48s
```

Fonte: elaborada pelos autores (2026).

O relatório HTML do Playwright consolida a execução por arquivo de teste, conforme a Figura 3.

Figura 3 — Relatório HTML do Playwright com os 7 testes aprovados

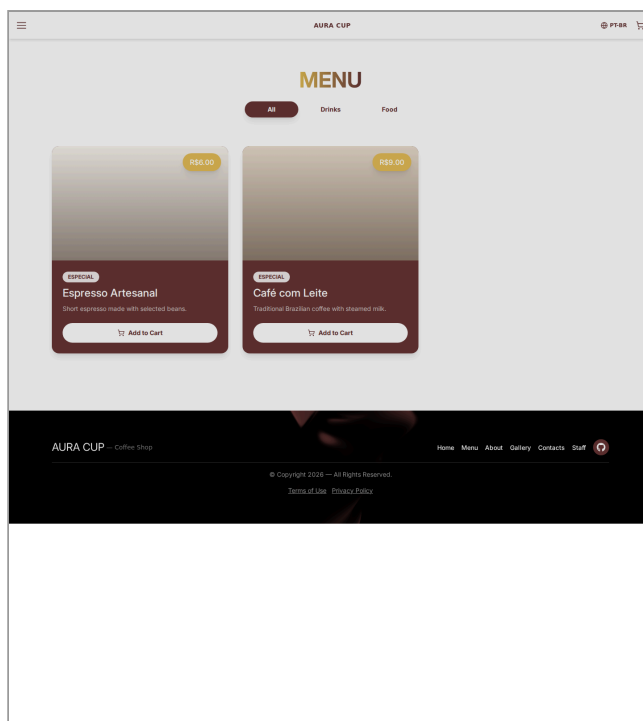


Search tests		All 7	Passed 7	Failed 0	Flaky 0	Skipped 0	⚙️
6/11/2026, 3:29:16 PM Total time: 1.4m							
▼ cardapio.spec.js							
✓	abre o menu e mostra os produtos	cardapio.spec.js:4	5.1s				
▼ pedido.spec.js							
✓	envia um pedido e exibe a confirmacao	pedido.spec.js:4	18.9s				
✓	bloqueia envio de pedido com numero de mesa invalido	pedido.spec.js:70	12.8s				
✓	bloqueia envio de pedido sem numero de mesa	pedido.spec.js:98	11.7s				
▼ seguranca.spec.js							
✓	nao executa conteudo malicioso vindo do cardapio	seguranca.spec.js:4	2.7s				
✓	abre a area restrita e lista pedidos para atendentes	seguranca.spec.js:35	14.5s				
✓	bloqueia acesso de atendente com credenciais invalidas	seguranca.spec.js:58	13.2s				

Fonte: elaborada pelos autores (2026).

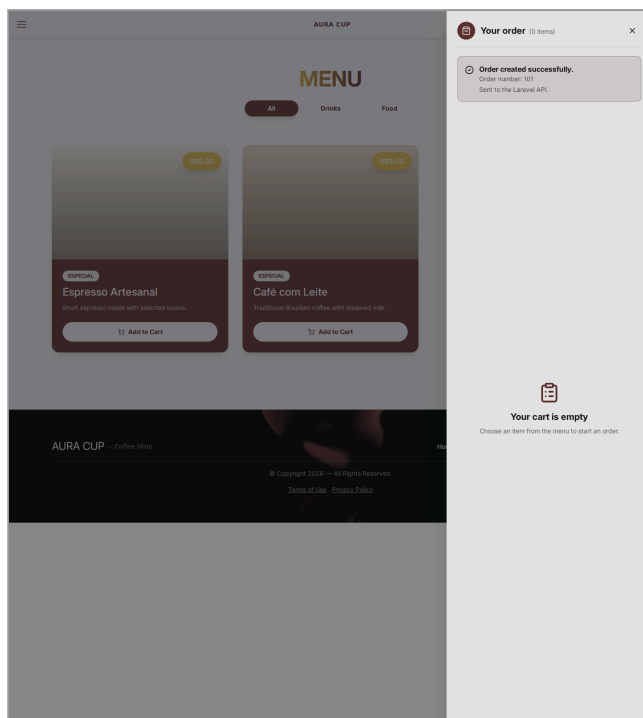
No fluxo completo do usuário (CT02), esperava-se montar o carrinho, ajustar a quantidade, informar a mesa e receber a confirmação do pedido; o resultado obtido correspondeu ao esperado, com o cardápio carregado pela API (Figura 4) e o pedido nº 101 confirmado na interface (Figura 5). A asserção sobre o payload enviado confirmou mesa 7 e dois itens do produto selecionado.

Figura 4 — Cardápio carregado com produtos da API



Fonte: elaborada pelos autores (2026).

Figura 5 — Confirmação do pedido ao final do fluxo completo do usuário



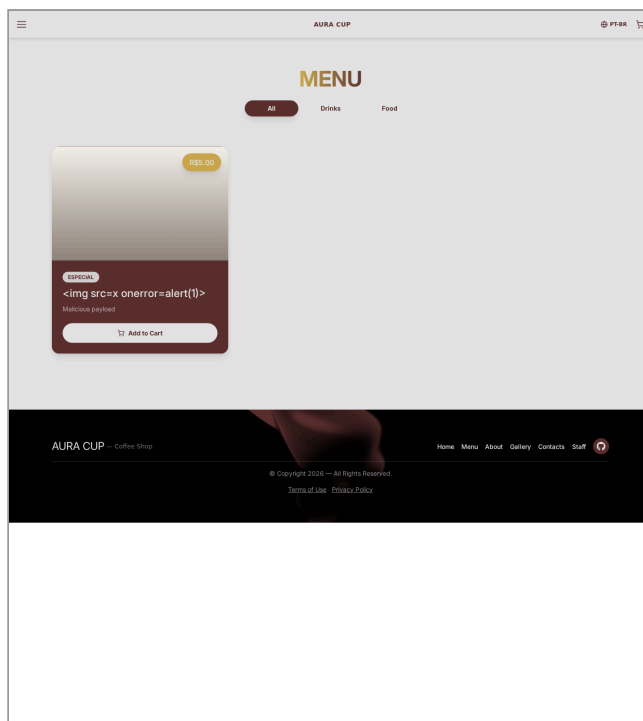
Fonte: elaborada pelos autores (2026).

Nos casos de validação (CT03 e CT11), esperava-se que mesa inválida ou vazia bloqueasse o envio; o resultado obtido confirmou que o navegador marca o campo como

inválido e que nenhuma requisição é enviada à API. Na API (CT07), payload inválido foi rejeitado com código 422 e mensagens de validação.

No teste de segurança (CT04), esperava-se que conteúdo malicioso vindo da API não executasse JavaScript; o resultado obtido mostrou o payload `<img src=x onerror=alert(1)>` renderizado como texto escapado, sem disparo de `alert` (Figura 6).

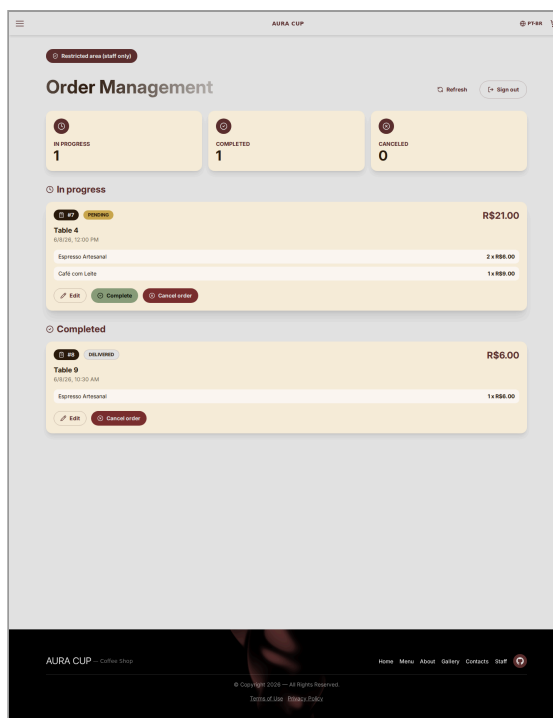
Figura 6 — Payload de XSS exibido como texto escapado, sem execução



Fonte: elaborada pelos autores (2026).

Na área restrita (CT05, CT06 e CT09), o login válido de atendente liberou a gestão de pedidos (Figura 7), enquanto credenciais inválidas e usuários sem perfil de atendente foram rejeitados com mensagem de erro e resposta 401, sem emissão de token.

Figura 7 — Gestão de pedidos liberada após autenticação do atendente



Fonte: elaborada pelos autores (2026).

Na integração com o banco de dados (CT08), um pedido real criado via POST /api/orders persistiu com total de R\$ 21,00, correspondente a dois Expressos Artesanais (R\$ 6,00) e um Café com Leite (R\$ 9,00), conforme as tabelas orders e order_items na Figura 8.

Figura 8 — Pedido persistido nas tabelas orders e order_items do SQLite



```

sqlite3 - pedido criado via POST /api/orders

$ sqlite3 database/database.sqlite

sqlite> SELECT id, table_number, status, total_price, created_at FROM orders;
id table_number status total_price created_at
-----
1 7 pendente 2100 2026-06-11 16:28:49

sqlite> SELECT order_id, product_id, quantity, price FROM order_items;
order_id product_id quantity price
-----
1 1 2 600
1 2 1 900

```

Fonte: elaborada pelos autores (2026).

6 FALHAS ENCONTRADAS E CORREÇÕES

Durante a preparação da suíte foram encontradas três falhas, todas corrigidas na versão final. A primeira e mais relevante foi a inconsistência entre os testes de API e o seeder do cardápio: os testes esperavam a categoria “Cafés Especiais” e produtos antigos, enquanto o sistema atual cadastra as categorias “Especial” e “Para Acompanhar” e o produto “Café com Leite”. A Figura 9 mostra a reprodução real da falha, com dois testes reprovados (Unable to find JSON fragment e ModelNotFoundException).

Figura 9 — Reprodução da falha de inconsistência entre testes e seeder (2 reprovados)

```

php artisan test --reproducao-da-falha-corrigida-versao-antiga-dos-testes

PASS Tests\Feature\AttendeeApiTest
  ✓ attendant can login and receive token 0.22s
  ✓ non attendant cannot login to attendant area 0.02s
  ✓ attendant orders are sorted with pending priority and time 0.03s
  ✓ attendant can edit complete and cancel order 0.03s

FAIL Tests\Feature\MenuApiTest
  ✗ categories endpoint lists seeded categories 0.02s
  ✓ products endpoint lists seeded products 0.01s
  ✓ product model accepts translated menu fields 0.01s

FAIL Tests\Feature\OrderApiTest
  ✗ order can be created with products and real total 0.01s
  ✓ order show returns products for existing order 0.01s
  ✓ order creation rejects invalid payload 0.01s

-----
FAILED Tests\Feature\MenuApiTest > categories endpoint lists seeded cate...
Unable to find JSON fragment:

[{"name": "Cafes Especiais"}]

within

[[{"created_at": "2026-06-11T16:26:46.000000Z", "id": 1, "name": "Especial", "updated_at": "2026-06-11T16:26:46.000000Z"}, {"created_at": "2026-06-11T16:26:46.000000Z", "id": 2, "name": "Para Acompanhar", "updated_at": "2026-06-11T16:26:46.000000Z"}]].
Failed asserting that false is true.

at tests\Feature\MenuApiTest.php:26
22 | (
23 |     $this->getJson('/api/categories')
24 |     ->assertOk()
25 |     ->assertJsonCount(2)
26 |     ->assertJsonFragment(['name' => 'Cafes Especiais'])
27 |     ->assertJsonFragment(['name' => 'Para Acompanhar'])
28 | )
29 |
30 | public function test_products_endpoint_lists_seeded_products(): void

-----
FAILED Tests\Feature\OrderApiTest > order can be... ModelNotFoundException
No query results for model [App\Models\Product].

at vendor/laravel/framework/src/Illuminate/Database/Eloquent/Builder.php:786
782 |         if ($model = $this->first($columns)) {
783 |             return $model;
784 |         }
785 |
786 |         throw (new ModelNotFoundException)->setModel($this->getModel());
787 |     }
788 |
789 |     /**
790 |      * Execute the query and get the first result or call a callback.

+1 vendor/frames
2 tests\Feature\OrderApiTest.php:26

Tests: 2 failed, 8 passed (34 assertions)
Duration: 0.55s

```

Fonte: elaborada pelos autores (2026).

A correção aplicada alinhou as expectativas dos testes ao cardápio real. As demais falhas foram: mocks do Playwright desatualizados em relação ao cardápio (produto e total divergentes), corrigidos em `helpers.js`; e ausência da pasta `backend/tests/Unit`, que causava falha estrutural do PHPUnit e foi adicionada ao repositório. A análise completa, com sintoma, resultado esperado, resultado obtido e correção de cada falha, está em `docs/evidencias-execucao/04-falhas/falhas-encontradas.md`.

Como recomendação de melhoria, sugere-se exibir mensagem de validação própria da aplicação para o campo de mesa (hoje a validação é a nativa do navegador) e padronizar os textos da interface em um único idioma.

7 CONCLUSÃO

Os resultados demonstram que o sistema AuraCup atende aos comportamentos esperados nos fluxos principais: o cardápio é carregado corretamente, o fluxo completo de pedido funciona de ponta a ponta, as validações de entrada bloqueiam dados inválidos, o conteúdo malicioso é neutralizado por escapamento e a área de atendentes é protegida por autenticação e autorização. A atividade também evidenciou o valor dos testes automatizados com asserções: as falhas de inconsistência entre testes e dados reais só foram detectadas porque cada caso valida o comportamento com `expect`, e não apenas executa ações na interface.

REFERÊNCIAS

LARAVEL. **Laravel Documentation:** testing. Disponível em: <https://laravel.com/docs/testing>. Acesso em: 11 jun. 2026.

MICROSOFT. **Playwright Documentation**. Disponível em: <https://playwright.dev/docs/intro>. Acesso em: 11 jun. 2026.

OWASP FOUNDATION. **Cross Site Scripting (XSS)**. Disponível em: <https://owasp.org/www-community/attacks/xss/>. Acesso em: 11 jun. 2026.

PHPUNIT. **PHPUnit Manual**. Disponível em: <https://docs.phpunit.de/>. Acesso em: 11 jun. 2026.